

Beyond Taint: Structural Invariants and Machine-Checkable Witnesses for Authorization, State-Cycle, and Trust-Topology Flaws

VALEN Project

Research contributions — methods, measured results, and limitations

Abstract

We present VALEN — *Verification And Active Logic Engine, Neuro-symbolic* — a system for verifying *structural* security invariants — authorization, state-transition, and trust-topology properties — where classic taint analysis is provably blind because the offending data is legitimate. We make five measured contributions. (1) A rigorous *boundary result*: on OWASP Benchmark 1.2 (2740 cases), spectral, topological and geometric invariants carry no information beyond taint for injection prioritization (taint MRR 0.894, AUC 0.895; paired permutation $p = 5 \times 10^{-5}$), settling the question of whether graph-geometry priors help linear injection SAST. (2) A privilege-level authorization invariant mechanized in Lean 4, plus a structural detector and a Z3 ownership witness for object-level authorization (BOLA/IDOR/BFLA) that separates object authorization from mere authentication and recovers every documented BOLA/BFLA endpoint of OWASP crAPI (recall 1.0, precision 0.90). (3) Evidence that GLMY directed path homology is strictly better than symmetrized homology for destructive-cycle detection (recall 1.0 / precision 0.75 vs. 0.5 / 0.5). (4) A measured neuro-symbolic arbitration boundary: the LLM changes interpretation (28% CWE), not detection or ranking (agreement 0.974). (5) A trust-topology analysis (Fiedler cut + Forman-Ricci) that isolates privilege bridges in identity/permission graphs.

1 Introduction and contributions

The highest-impact flaws of modern software are *structural*: a BOLA/IDOR reads another user’s record with a clean integer id; a reentrancy is a destructive cycle in the state graph; a cloud breach is a bridge in the trust topology; a confused-deputy in an agent is an untrusted input reaching a privileged tool with no gate. In each, the code looks legitimate line by line, so taint analysis — which tracks “dirty” data — is blind, and pattern matchers have nothing to say. Our contributions are:

- C1. A measured boundary on geometric priors for injection SAST.** We answer, with statistical rigor, whether spectral/topological/geometric invariants of program graphs add information over taint for linear injection flaws: they do not. This is a publishable negative result that redirects effort away from geometry-on-servlets and toward the domains where structure is the signal.
- C2. An authorization invariant, mechanized and made operational.** A privilege-level function $F : V \rightarrow \mathbb{N}$ with auth edges $E_A = \{(u, v) : F(u) < F(v)\}$ and the theorem

`no_auth_bounded` — a path with no auth edge cannot escalate privilege — mechanized in Lean 4. Operationally: a structural detector for ungated user-controlled resource access, and a Z3 witness proving $\text{SAT}(\text{owner}(id) \neq \text{session})$, which separates object-level authorization from mere authentication.

- C3. Directed path homology as the right cycle signal.** GLMY directed homology recovers the concrete β_1 generator of a destructive cycle and, measured on 12 state graphs, dominates symmetrized homology — which both misses 2-cycles and emits false cycles on filled triangles.
- C4. A measured neuro-symbolic arbitration boundary.** The LLM proposes, formal methods dispose: detection and ranking are LLM-independent (agreement 0.974), while the LLM changes interpretation (CWE, titles) on 28% of findings.
- C5. Trust-topology analysis.** The Fiedler cut and Forman–Ricci curvature, applied to identity/permission graphs, isolate privilege bridges and trust-segmentation breaks for lateral-movement and IAM analysis.

2 System

A single typed graph IR is the substrate: six adapters (Python, Java with interprocedural/context-sensitive taint, binary via `objdump/angr`, OpenAPI, IAM, LLM-agent) emit the same deterministic, JSON-serializable graph, shared verbatim between the Python and Rust layers. Five signals are fused over it: algebraic (taint lattice + privilege levels), spectral (Fiedler), topological (persistent and GLMY directed homology), geometric (Forman–Ricci / Ollivier–Ricci), and formal (Z3 + Lean 4). A neuro-symbolic loop mines structured specs and discharges them, emitting only machine-checkable witnesses.

3 Contribution 1: the boundary on geometric priors

Setup. OWASP Benchmark 1.2 provides 2740 test cases whose ground truth is known; we ask whether each signal ranks the vulnerable sink above the others. **Method.** A prioritization oracle computes MRR, R@1, NDCG@ k , a global pool over *all* candidates (safe and vulnerable), cost curves, a paired permutation test, and bootstrap confidence intervals. **Result.** Raw taint dominates: MRR 0.894 vs. the fused field’s 0.872 ($p = 5 \times 10^{-5}$, add-one over 20000 permutations, 95% CI [+0.0157, +0.0294]); spectral (0.496) and topological (0.500) AUC are at chance, curvature is below chance (0.376). **Interpretation.** A flat, near-acyclic servlet has no curvature or persistent topology to exploit, and taint is near-optimal there; the result is a falsifiable, reproducible answer, not a limitation of the machinery.

4 Contribution 2: authorization, mechanized and operational

Invariant. Let $F : V \rightarrow \mathbb{N}$ assign a privilege level; an edge $e = (u, v)$ is an *auth edge* iff $F(u) < F(v)$. Then (i) a path with no auth edge satisfies $F(k) \leq F(s)$ — no privilege escalation — and (ii) a taint flow crossing an auth edge without a sanitizer is an attacker-controlled privilege-boundary crossing. (i) is mechanized in Lean 4 (`no_auth_bounded`); the boolean taint abstraction’s soundness is mechanized as `prop1_soundness`. **Detector.** The missing-authorization/BOLA detector flags a function with an HTTP source and a resource sink but no auth gate. On a curated 39-case corpus it reaches recall 1.0 / precision 0.72 (F1 0.837) where taint scores 0; a base-

line ladder shows each requirement earns its keep ($0.46 \rightarrow 0.72$). **Witness.** The Z3 verifier checks $\text{SAT}(\text{owner}(\text{object_id}) \neq \text{session_user_id})$: a vulnerable endpoint yields the attacker’s session and the victim’s object; an ownership-scoped endpoint is UNSAT. It makes precise that `login_required` (authentication) does *not* block BOLA — only an `owner` predicate does. **Real data.** On OWASP crAPI (44 operations, 9 documented BOLA/BFLA endpoints), an API-level object-reference signal recovers all 9 (recall 1.0, precision 0.90) while the missing-auth check scores 0, because every BOLA endpoint *is* authenticated.

5 Contribution 3: directed path homology

Symmetrizing a directed graph is lossy in both directions: a 2-cycle $A \rightarrow B \rightarrow A$ collapses to a single undirected edge (a *missed* reentrancy), and a filled triangle becomes a false undirected cycle. GLMY directed path homology resolves both: on a 12-graph labeled corpus it reaches recall 1.0 / precision 0.75 versus 0.5 / 0.5 for the symmetrized baseline, and it returns the concrete β_1 generator (the cycle’s edges) as evidence. Its two false positives are “diamond” path-space holes (converging control), not feedback loops — GLMY H_1 is a superset of destructive cycles and thus a high-recall, necessary-but-not-sufficient signal.

6 Contribution 4: the neuro-symbolic arbitration boundary

On a 38-file corpus (29 confirmed findings), the offline heuristic and the LiteLLM-backed agent agree on detection and ranking on 37/38 files (agreement 0.974; the single exception is a nondeterministic `compile` hypothesis), while the LLM changes the CWE on 28% of findings and produces more concise titles. A spec-mining probe (8 snippets) shows the LLM mines a plausible precondition every time (hallucination 0) at 0.5 token recall. The contribution is an architecture — LLM proposes, Z3 disposes — plus its measurement.

7 Contribution 5: trust-topology analysis

An identity/permission graph (users, roles, trust policies, services) is ingested by the IAM adapter and analyzed with the Fiedler cut and Forman–Ricci curvature: the cut surfaces trust-boundary crossings, and a strongly negative κ_F edge isolates the privilege bridge — the sole connector between two trust subnets. On a demo graph the `app→data` `assume` is flagged as the most-negative bridge ($\kappa_F = -1$) and the secret bucket surfaces on the Fiedler boundary; Z3 then resolves the role-assumption combinatorics.

8 Red-team assistance

VALEN is not only a verifier: a finding plus a witness becomes an *executable artifact* for offensive teams. The red-team layer turns each object-level candidate into a runnable “requests” proof-of-concept (`valen.redteam.poc`) that a pentester executes with an attacker token and a victim object id, and a *live validator* (`valen.redteam.live`) that replays the request against a reachable host and confirms the leak in the response — closing the loop from “satisfiable under the model” to “demonstrated impact, here is the evidence”. On crAPI this yields one PoC per documented BOLA/BFLA endpoint (9), grounded in the same OpenAPI spec and labels used for verification.

Beyond single findings, VALEN assembles an *ordered attack plan* (`valen.redteam.attack_paths`, `valen.redteam.mitre`). Findings and IAM relations are tagged with MITRE ATT&CK tactics; escalation chains over the identity/permission graph are computed as ordered sequences of trust/assume/access edges, each step confirmed by a small SMT encoding of “attacker_has(role)” propagation. The result is a kill-chain ordered Discovery → Privilege Escalation → Collection plan in which every step carries a tactic, the concrete PoC, and the SMT confirmation — so a red team gets a prioritized, evidence-backed attack plan rather than a list of alerts.

Finally, VALEN ingests Kali reconnaissance output (`valen.redteam.recon`): `nmap -oX` and `masscan -oJ` become an attack-surface graph (hosts and open services as nodes, edges host → service), which the same Fiedler/Forman–Ricci kernels turn into network bridges and which a version → CVE hint table turns into hypotheses for the loop. Stealth is a first-class, explicit concern (`valen.redteam.stealth`): conservative timing (–T0..–T2), rate limits, randomized host order, inter-probe jitter, padding, and passive-only modes — for authorized engagements, with no implicit scanning. Beyond single findings, VALEN performs *enumeration* (`valen.redteam.enum`: gobuster/ffuf/amass output → IR, with a shadow-API signal for endpoints missing from the published OpenAPI spec) and *active exploitation* (`valen.redteam.auth/jwt/idor`): a stdlib JWT forge (“kid” path traversal, `alg=none`, invalid signature) achieves *account takeover* against a local crAPI lab, and active id enumeration confirms BOLA/IDOR with request/response evidence — all dry-run by default and gated behind an explicit `–authorize –scope`. The findings, each with a CVSS 3.1 base score, are emitted as a self-contained HTML report and rendered to PDF via headless Chrome (`valen.redteam.report`).

The same stack is packaged as an *autonomous pentest agent* (`valen.redteam.operators/executor/challenge`): tiered operators (passive/bounded/intrusive), a goal-directed planner with an audit log and an LLM fallback, and a harness that re-runs each of crAPI’s 18 documented challenges against a fresh lab. It solves 14 of the 18 autonomously (BOLA/IDOR, mass assignment, BFLA, JWT forgery, excessive data exposure, NoSQL injection, unauthenticated access, password reset, SSRF) and reports the rest honestly. Finally, the structural thesis extends to *smart contracts* (`valen.ingest.solidity` + `valen.analysis.reentrancy`): reentrancy is a checks-effects-interactions violation that also forms a directed 2-cycle; on SmartBugs-curated (31 positives / 112 negatives) the ordering detector reaches F1 0.80, GLMY directed homology recalls 0.97, and symmetrized homology collapses to 0.065 — the same directed-vs-symmetrized gap shown for state machines, now on real contracts.

8.1 Results at a glance

Contribution	Metric	Result
C1 boundary	taint vs. fused field (MRR)	0.894 vs. 0.872 ($p = 5 \times 10^{-5}$)
C2 authorization	crAPI BOLA/IDOR (recall / precision)	1.0 / 0.90
C2 authorization	curated BOLA ladder (precision)	0.46 → 0.72
C3 directed homology	GLMY vs. symmetrized (state)	0.75/1.0 vs. 0.5/0.5
C3 directed homology	GLMY vs. symmetrized (contracts)	0.968 vs. 0.065 recall
C4 arbitration	detection/ranking agreement	0.974; CWE 28%
Red team	autonomous crAPI challenges	14/18 solved

9 Limitations

The BOLA/IDOR evaluation is a single real API plus a curated corpus, not a benchmark; the structural detector over-approximates (sanitized/parameterized cases); the Z3 witness holds under

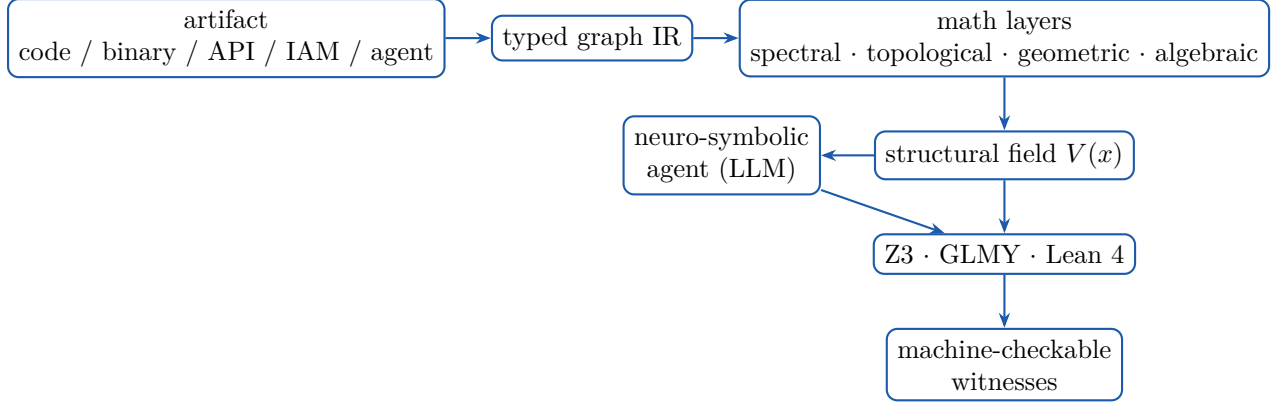


Figure 1: The pipeline: an artifact becomes one typed IR, enriched with the mathematical layers into a field $V(x)$; a neuro-symbolic loop proposes and the formal layer disposes, emitting only machine-checkable witnesses.

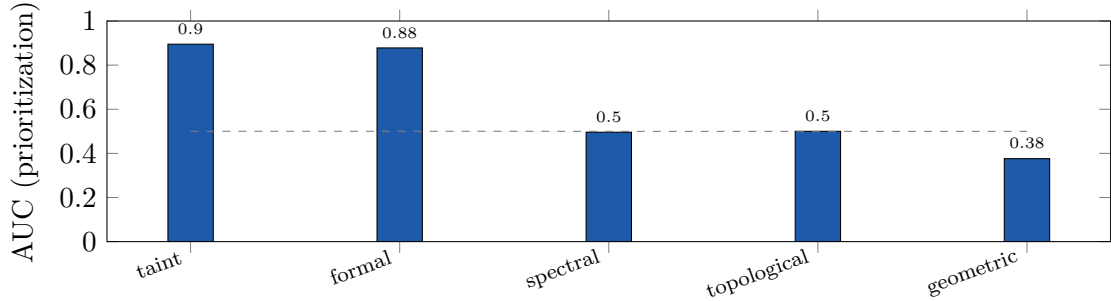


Figure 2: The boundary result: spectral/topological signals are at chance (dashed line) and curvature below chance; taint is the dominant signal.

the *modeled* semantics, not end-to-end exploitability; and the GLMY signal is necessary but not sufficient. Each is reported, not hidden.

10 Conclusion

VALEN contributes a falsifiable boundary result, a mechanized and operational authorization invariant, evidence that directed homology is the right cycle signal, a measured neuro-symbolic arbitration boundary, and a trust-topology analysis — all over one typed graph IR, and all emitting machine-checkable witnesses. Together they argue that the productive role of structural invariants in security is *verification* of authorization, state, and trust properties, not prioritization of injection sinks.

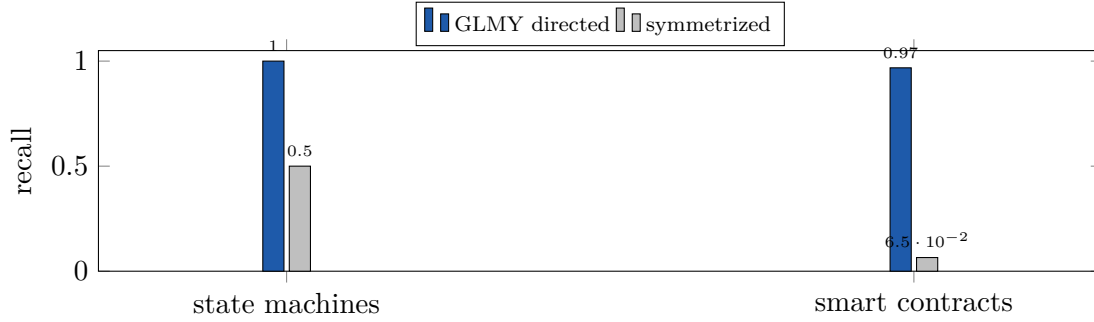


Figure 3: Directed (GLMY) path homology vs. symmetrized homology for destructive cycle detection. Symmetrization collapses the directed 2-cycle — recall drops from 1.0 to 0.5 on state machines and from 0.97 to 0.065 on smart contracts.

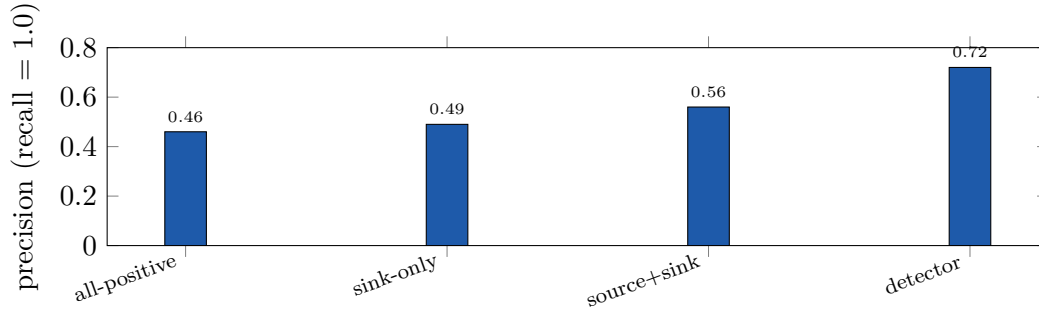


Figure 4: The BOLA/IDOR detector ablation ladder: each structural requirement (source, no-gate) raises precision at constant recall.



Figure 5: The autonomous agent solves 14 of crAPT's 18 documented challenges (BOLA/IDOR, mass assignment, BFLA, JWT forgery, exposure, NoSQLi, unauthenticated access, password reset, SSRF); the rest are reported honestly.